

新IR 学习文档

目录

- 1. IR理解：
 - 1.1编译器IR
 - 1.2 机器学习框架IR
- 参考链接：
- 2 新IR 解决的问题
- 3 新IR 解决此类问题的方法
- 4 新IR 设计内容
 - 4.1IR整体变化：类型-模型结构-高阶特性-pass
 - 4.2 数据结构：
 - 整体：
 - 4.2.1 类型设计：
 - 4.2.2计算图设计
 - 4.2.3 算子类型信息

1. IR理解：

https://openmlsys.github.io/chapter_frontend_and_ir/intermediate_representation.html

IR 为 中间表达，描述了程序结构的抽象特点，需要简洁的表达足够的信息，以向下传递信息，进行分析和优化。

1.1编译器IR

编译器IR 功能：编译器IR用于表示源代码的数据结构或代码，是程序编译过程中介于源语言和目标语言之间的程序表示。几乎所有的编译器都需要某种形式的中间表示，来对被分析、转换和优化的代码进行建模分类：

编译器IR分类：

1	Linear IR	基于线性代码	堆栈机代码、三地址代码
2	Graphical IR	基于图	抽象语法树(Abstract Syntax Tree, AST)、有向无环图(Directed Acyc
	Hybrid IR	基于图与线性代	LLVM IR

LLVM IR使用线性中间表示表示基本块，使用图中间表示表示这些块之间的控制流，如 [图 6.3.4](#) 所示。基本块中，每条指令以静态单赋值(Static Single Assignment, SSA) [Richard1995A] 形式呈现，这些指令构成一个指令线性列表。**SSA形式要求每个变量只赋值一次，并且每个变量在使用之前定义。**控制流图中，每个节点为一个基本块，基本块之间通过边实现控制转移。

1.2 机器学习框架IR

- 1) **张量表达**。机器学习框架主要处理张量数据，因此正确处理张量数据类型是机器学习框架中间表示的基本要求。
- 2) **自动微分**。自动微分是指对网络模型的自动求导，通过梯度指导对网络权重的优化。主流机器学习框架都提供了自动微分的功能，在设计中间表示时需要考虑自动微分实现的简洁性、性能以及高阶微分的扩展能力。
- 3) **计算图模式**。主流机器学习框架如TensorFlow、PyTorch、MindSpore等都提供了**静态图**和**动态图**两种计算图模式，静态计算图模式先创建定义计算图，再显式执行，有利于对计算图进行优化，高效但不灵活。动态计算图模式则是每使用一个算子后，该算子会在计算图中立即执行得到结果，使用灵活、便于调试，但运行速度较低。机器学习框架的中间表示设计同时支持静态图和动态图，可以针对待解决的任务需求，选择合适的模式构建算法模型。
- 4) **支持高阶函数和闭包** [2010C]。高阶函数和闭包是函数式编程的重要特性，高阶函数是指使用其它函数作为参数、或者返回一个函数作为结果的函数，闭包是指代码块和作用域环境的结合，可以在另一个作用域中调用一个函数的内部函数，并访问到该函数作用域中的成员。支持高阶函数和闭包，可以抽象通用问题、减少重复代码、提升框架表达的灵活性和简洁性。
- 5) **编译优化**。机器学习框架的编译优化主要包括**硬件无关的优化**、**硬件相关的优化**、**部署推理相关的优化**等，这些优化都依赖于中间表示的实现。
- 6) **JIT(Just In Time)能力**。机器学习框架进行编译执行加速时，经常用到JIT即时编译。**JIT编译优化**将会对中间表示中的数据流图的可优化部分实施优化，包括循环展开、融合、内联等。中间表示设计是否合理，将会影响机器学习框架的JIT编译性能和程序的运行能力。

例：

PyTorch框架采用命令式编程方式，其TorchScript IR以基于SSA的线性IR为基本组成形式，并通过JIT即时编译的Tracing和Scripting两种方法将Python代码转换成TorchScript IR。如下Python代码使用了Scripting方法并打印其对应的中间表示图：

</>

C++ | 收起 ^

```
1 import torch
```

```
2 @torch.jit.script
3 def test_func(input):
4     rv=10.0
5     for i in range(5):
6         rv=rv+input
7         rv=rv/2
8     return rv
9
10 print(test_func.graph)
```

Jax机器学习框架同时支持静态图和动态图，其中间表示采用Jaxpr(JAX Program Representation) IR。Jaxpr IR是一种强类型、纯函数的中间表示，其输入、输出都带有类型信息，函数输出只依赖输入，不依赖全局变量。

Jaxpr IR的表达采用ANF(A-norm Form)函数式表达形式，ANF文法如下所示：

```
</> C++ | 收起 ^
1 <aexp> ::=  NUMBER | STRING | VAR | BOOLEAN | PRIMOP
2           |  (lambda (VAR ...) <exp>)
3 <cexp> ::=  (<aexp> <aexp> ...)
4           |  (if <aexp> <exp> <exp>)
5 <exp> ::=  (let ([VAR <cexp>]) <exp>) | <cexp> | <aexp>
```

ANF形式将表达式划分为两类：原子表达式(aexp)和复合表达式(cexp)。原子表达式用于表示常数、变量、原语、匿名函数，复合表达式由多个原子表达式组成，可看作一个匿名函数或原语函数调用，组合的第一个输入是调用的函数，其余输入是调用的参数。如下代码打印了一个函数对应的JaxPr：

```
</> C++ | 收起 ^
1 from jax import make_jaxpr
2 import jax.numpy as jnp
3
4 def test_func(x, y):
5     ret = x + jnp.sin(y) * 3
6     return jnp.sum(ret)
7
8 print(make_jaxpr(test_func)(jnp.zeros(8), jnp.ones(8)))
```

其对应的JaxPr为：

```
</> C++ | 收起 ^
```

```
1 { lambda ; a:f32[8] b:f32[8]. let
2   c:f32[8] = sin b
3   d:f32[8] = mul c 3.0
4   e:f32[8] = add a d
5   f:f32[] = reduce_sum[axes=(0,)] e
6   in (f,) }
```

Jax框架结合了Autograd 和 JIT，基于Jaxpr IR，支持循环、分支、递归、闭包函数求导以及三阶求导，并且支持自动微分的反向传播和前向传播。

参考链接：

【源码学习】IR 设计和底层数据结构

2 新IR 解决的问题

1. 目前框架有两套IR program 和 graph 功能有重复，且各有缺点无法满足新的需求

分布式：算子中加分布式属性 (id, TensorDistAttr, OperatorDistAttr); 动态维度 (batch, 其他) -- 动态语意抽象，定制算子属性描述动态语义

编译器：复用pass 优化 -- 统一编译器和框架的描述（类型，算子，模型）

推理：保证模型结构无环，pass 简洁完备 -- 数据结构保证无环，升级算子定义保证inplace语意的表达。

框架：支持list, dict, 嵌套数据结构；复杂ir 分析（控制流分析，依赖分析）

其他：

1. 原有数据结构program_desc&block_desc&op_desc&var_desc 内嵌 proto 的desc; graph 内嵌 program_desc 同步成本大且不够鲁棒。

2. 原有program 中 var 和 op 的定义解耦，无法保证计算图无环，在pass 中对算子顺序的调整会导致出错；执行器中进行多流分配会导致出错。且想要知道var的关联Op, 需要遍历block (在backward中剪枝逻辑中出现过) 进行字符串比对，原有graph中变量和算子同时作为节点，图优化不易。

3. OpInfo 中保存算子的抽象信息，但很多算子不需要信息的全集，使得opinfo 结构臃肿，但单个算子信息量没有增加 -- 设计Opinfo数据结构，新增算子场景不影响无关算子

4. 对于输入输出是相同变量的算子，原有框架通过给算子赋相同变量名标志inplace, 参数更新算子，而share_data的view类算子，输入输出又是不同变量名，没有统一标准描述此类算子。在图变换和多流并发时容易使内存的改动不满足程序顺序而出错。--设计别名算子定义方式，保证图变换和并行的安全性

3 新IR 解决此类问题的方法

4 新IR 设计内容

4.1 IR整体变化：类型-模型结构-高阶特性-pass

1. 构建可扩展的类型系统。(Type)
 - a. 能够支持List、Dict等复杂容器
 - b. 支持类型的嵌套递归定义
2. 构建符合SSA的模型结构表示。(Program)
 - a. 模型结构无环
 - b. Program&Graph合二为一。
3. 支持模型的高阶特性。(复杂算子和算子分层分类)
 - a. 支持通过dialect对算子进行分层、分类以及扩展。
 - b. 支持控制流算子、inplace算子、编译器算子等复杂算子的合理定义。
 - c. 合理支持多层控制流嵌套。
4. 构建可复用的Pass设施。(Pass)
 - a. 开发能够兼顾推理、分布式、编译器的Pass基础设施。
 - b. Pass更加完备、稳定、易开发。

4.2 数据结构：

整体：

Type 类型对象(impl 指针)；

Attribute表示属性对象

Value 变量（包含value的创建算子，使用算子list）；

Operation 算子（包含输入vector，输出vector，名称string）；

OpOperand 算子输入，OpResult 算子输出，OpInfo 算子类型信息，Attribute 属性（编译时常量）；

Block（list<Operation>）基本块，

region（vector<Block>），基本块组成的闭包，

program 模型 list(Operation)

与原有数据结构对应 *operation - op* , *value - var_desc*

4.2.1 类型设计：

原有：proto 的var_desc , c++ 模版索引 数字标识， phi 算子库枚举； 类型转换通过switch语句
prototype-vartype-phitype

现有：

Type 的唯一标识为 **TypeId**,

AbstractType 为Type的抽象类，有**name**, **typeid** 属性

IRContext中有以TypeId为key的散列表 `std::unordered_map<TypeId, AbstractType*>`

`register_types`。记录框架支持的所有Type类,以及相应的性质

及 `StorageUniquer type_storage` 作为IRContext 的属性做内存管理

Type 类只有 `TypeStorage* impl` 成员，指向ircontext中的内存地址，内存是否相同标志着Type对象是否相等

TypeStorage里面存储所有Type对象都需要的信息 `AbstractType *abstract_type`, 这个指针和IRContext中的`register_types`的哈希对象共享底层。

类型:

无参Type, 对应唯一的Type 对象， eg: `Float32Type`, `Float64Type`, 用TypeStorage存储

有参Type ,一般对应多个对象，需要继承TypeStorage增加参数值，

eg: `DenseTensorTypeStorage`, 新增了`data_type`、`dims`和`lod_level`三个成员变量存储参数，

`DenseTensorType`类 对应的Type对象的impl指针，指向的是TypeStorage的真实类型是

`DenseTensorTypeStorage`

所有Type类的派生，只派生接口，不派生成员，保证，从子类到父类的类型转换，不会丢失任何信息

当定义一种具体类型（`ConcreteType`）时，需要考虑它的基类（`BaseType`）以及相应的内存类型（`StorageType`）。通过提供一个TypeBase的模版工具类将这三者关联起来

</>

C++ | 收起 ^

```
1 class Float32Type : public TypeBase<Float32Type, Type, TypeStorage> {
2     public:
3     // 该函数会返回一个Float32Type对象，由于集成关系，可以直接转变为Type对象。
4     // 该Type对象的impl指针是恒定且唯一的。
5     static Float32Type get(TypeContext *context);
6 }
7
8 // 将该Type对应的TypeID注册到TypeContext中。
9 REGISTER_TYPE_ID(Float32Type)
```

对原有系统改动

1. `VarDesc` 中删除 `proto::VarDesc` 成员 避免同步，把其中的内容作为 `VarDesc` 的成员,删除相关更新同步的接口，增加序列化/反序列化接口 负责 `proto::VarDesc` 和 `VarDesc` 的转换。

2. is_persistable 属性移到persist OP (产生persistable var的 OP)
3. need_check_feed 属性移到 feedOp (产生feed var的OP)
4. is_parameter 属性 移到ParameterOp (产生Parameter的OP)
5. 分布式属性 ...

一个变量将会来自多个Op?

2.ProgramDesc、BlockDesc、OpDesc 中与proto 数据结构的嵌套替换为c++的数据结构, 删除 flush 接口

3.关于同步时的hash 增加need_rehash_标志变量, 4个desc有变化时标志为True, 获取hash 值时根据此标志判断是否需要rehash, rehash 后将其设为false。

4.类型系统会提供相应的python的接口。并逐步删除掉python中对proto::OpDesc、proto::BlockDesc、proto::VarDdesc、proto::OpDesc以及一些Type的使用。

5.类型系统会逐步统一paddle中所有对类型的使用。包括 proto中的AttrType、Variable中的Type、phi算子库中的type等等。

是否可以理解要删除proto, 原始的OP定义会放在哪里?

4.2.2计算图设计

计算图的有向边是变量(Value), 节点是算子(Operation)。算子信息分为四部分: 输入(OpOperandImpl)、输出(OpResultImpl)、属性(Attribute)、类型信息(OpInfo)。

算子信息都由xxximpl指针作为唯一成员, 便于隔离实现和调用, 方便重构。

OpInfo设计

OpInfo是类型信息,指同类算子的完全一致的信息, 由IRContext 管理(创建回收) 算子自身构造是接受OpInfo (OpInfoimpl 指针) 作为参数

attribute设计

attribute只描述常量, 若其即可成为常量又可以成为变量, 则定义为输入, 当为常量时插入 constOp将常量转换为变量。

定义工具类AttrBase将ConcreteAttribute, BaseAttribute, StorageAttribute关联

对每种Attribute类型, 需要首先在AttributeStorage基础上派生它的存储对象。eg class StrAttributeStorage : public AttributeStorage{}

Attribute的内存统一由IRContext管理, 多个场景公用。对于单个Attribute而言, 它的底层内存是只读的。用户不能修改StrAttribute的具体值, 只可以获取一个新的StrAttribute

value设计

只有对所有变量都有意义的信息才存储在变量中（类型，Use-Define链（获取定义该变量的唯一算子和使用该变量的算子列表）

算子负责自己定义变量的生命周期。（若该算子被析构，它创建的变量也会同时析构吗，若析构之后，该变量作为其他算子的输入被用到怎么办？）

考虑到OpOperandImpl(里面只包含4个指针)是8字节对齐的，因此OpOperand的低三位一定为0，用低三位来存储index。

为了支持输出个数大于8的算子，将输出分为内置输出(OpInlineResultImpl)和外置输出(OpOutOfLineResultImpl)，外置输出额外新增整型成员index表示下标。对OpResultImpl而言，低三位在0~5之间，表示该输出的真实类型是内置输出，为6表示该输出的真实类型是外置输出，7保留，方便以后扩展。

</>

C++ | 收起 ^

```
1 class ValueImpl {
2     protected:
3         // 只允许通过派生类来构造
4         ValueImpl::ValueImpl(Type type, int32_t index_): type_(type),
            first_user_(nullptr), index_(index){}
5         // 获取该变量的index，只允许派生类调用
6         int32_t index();
7         Type type_; // 变量的类型。
8         // OpOperandImpl一定是8字节对齐的，因此OpOperand低三位一定为0。低三位用来表示
            index_
9         // index_ 0~5表示前六个输出， 6表示第七个以及第七个之外的输出， 7保留，留作扩
            展。
10        OpOperand first_user_; //第一个使用该变量的输入。
11 };
12 class Value {
13     private:
14         ValueImpl* impl;
15 };
16 class OpResultImpl: public ValueImpl{};
17 class OpInlineResultImpl: public OpResultImpl{};
18 class OpOutOfLineResultImpl: public OpResultImpl{
19     private:
20         // 新增index_成员变量，因为此时，基类ValueImpl里面的index一定为6.
21         uint32_t index_;
22 };
23 // OpResult表示Op的输出，可以随遇拷贝
24 class OpResult : public Value {};
25 class OpOperandImpl{
26     private:
```



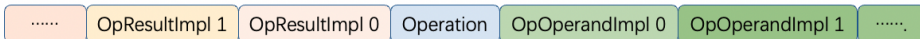
```

27     OpResult    source_; //该输入使用的变量。
28     OpOperand   next_user_; //使用同来源的下一个操作数。
29     OpOperand*  back_addr_; // 使用同来源的上一个操作数的地址。
30     Operation*  owner_;
31     // 构造函数私有
32     OpOperandImpl::OpOperandImpl(OpResult source, Operation* owner):
source_(source), owner_(owner){
33         back_addr_ = &source->first_user_;
34         next_user_ = source->first_user_;
35         if(next_user_) {
36             next_user_->back_addr = &next_user_;
37         }
38         source->first_user_ = this;
39     }
40 };
41 class OpOperand {
42     OpOperandImpl * impl_;
43 };
44

```

Operation 设计

Operation的对一个算子的具体描述，涉及到四个成员：输入、输出、属性以及类型信息：输入和输出的数量在构造时确定，构造完以后，数量不会改变。因此，在开辟Operation的时候，额外多开辟内存，前面存放输出，后面存放输入。



属性是一个DictionaryAttribute，它和类型信息（OpInfo）一样，都是指针的封装，只需要作为成员变量

将Operation指针进一步装饰为OpBase, 做具体算子的基类，并实现从Operation*到具体Op的转换

</>

C++ | 收起 ^

```

1 class Operation {
2     public:
3     // 创建一个算子.
4     static Operation *create(const std::vector<OpResult>& inputs, const
std::vector<Type>& output_types,
5                             DictionaryAttribute attrs, OpInfo info) {
6         // 先通过malloc函数开辟 输入、输出以及Operation所需的内存
7         // 然后再每个位置显式构造对象
8     }
9     destroy(Operation * op) { // 析构算子，释放内存}

```

```

10 private:
11     Operation(...); // 构造函数私有
12     ~Operation(); // 析构函数私有
13     // 属性。
14     DictionaryAttribute atts_;
15     // 算子类型信息
16     OpInfo info_;
17     // 输出个数
18     uint8_t num_results_;
19     // 输入个数
20     uint8_t num_operands_;
21 }
22 class OpBase {
23 public:
24     // 能够隐式转换为Operation*.
25     operator Operation *() const { return operation_; }
26     // 重载 ->运算符.
27     Operation *operator->() const { return operation_; }
28 protected:
29     explicit OpBase(Operation * operation) : operation_(operation) {}
30 private:
31     Operation * operation_;
32 }

```

权重Variable设计:

权重是特殊属性，数据量大，不易带在模型中hash 和判等，因此单独存放。在每个模型中，维护一个哈希表： `hash_map<StrAttribute, Variable*>`来表示该模型对应的权重值：包含

1. Type `type_`：类型；
2. void* `data_`：指向具体的数据的指针；
3. bool `is_mutable_`：表明数据是否会在模型的执行当中被改变；
4. 数据的大小、对齐等其他性质。

定义 `GetParameterOp`、`SetParameterOp`。分别从相应模型的哈希表中，获取、设置相应的权重内容。用于startup program 创建参数和 main_program获取参数，移动hash 表以进行通信。

定义`Get/SetCombineParameterOp`等，一次性加载&存储大批量权重。

模型导出的时候，会将模型中的哈希表存储为权重文件，算子列表存储为模型文件。

当从文件中初始化模型的时候，会将所有的Variable的`isMutable`设为False，然后遍历模型中的所有算子，遇见`SetParameterOp`的时候，就将相应的`isMutable`设为True。对于Pass而言，对于权重，可以通过访问相应的`isMutable`来判定是否可以将该Parameter当作常量进行变换。

4.2.3 算子类型信息

特征Trait和接口Interface 对算子进行另一维度的划分，对算子所拥有的类型信息抽象描述

特征：与具体算子无关，不需要多态。eg ReadOnly

接口：与算子类型相关，需要多态。eg infershape

</>

C++ | 收起 ^

```
1 using TraitId = TypeID;
2 template<class ConcreteTrait>
3 class OpTraitBase : public OpBase{
4     // 所有Trait的模版基类
5 public:
6     OpTraitBase::OpTraitBase(Operation* op): OpBase(op){}
7     TraitId getTraitId () {return TraitId::get<ConcreteTrait>();}
8 };
9
10 // Trait的定义
11 class ReadOnlyTrait: public OpTraitBase<ReadOnlyTrait>{};
12
13 using InterfaceId = TypeID;
14 template<class ConcreteInterface>
15 class OpInterfaceBase : public OpBase{
16     // 所有Interface的模版基类
17 public:
18     InterfaceId getInterfaceId () {return
19     InterfaceId::get<ConcreteInterface>();}
20 protected:
21     ConcreteInterface::Concept* impl_;
22     OpInterfaceBase::OpInterfaceBase(Operation* op,
23     ConcreteInterface::Concept* impl): OpBase(op), impl_(impl){}
24 };
25 // 需要具体算子类型的特征类,必须定义Impl结构体
26 class InferShapeInterface : public OpInterfaceBase<InferShapeInterface>{
27     struct Concept {
28         void(*infer_shape_) (Operation*);
29     };
30     //不允许定义新成员, 不允许自定义析构函数
31     template<class ConcreteOp>
32     struct Model: public Concept{
33         Model():Concept(InferShape){
34             assert(sizeof(Model) == sizeof(Concept));
35         };
36     };
37     static void InferShape(Operation* op) {
```

```

36         ConcreteOp concret_op = dyn_cast<ConcreteOp>(op);
37         assert (concret_op != nullptr);
38         concret_op.InferShape();
39     }
40 };
41
42 void InferShape() {
43     impl_>infer_shape_(operation_);
44 }
45 InferShapeInterface(Operation* op, Concept* impl):
46 OpInterfaceBase(op,impl){};
47 };

```

OpInfoImpl设计

</> OpInfoImpl的构建方式

C++ | 收起 ^

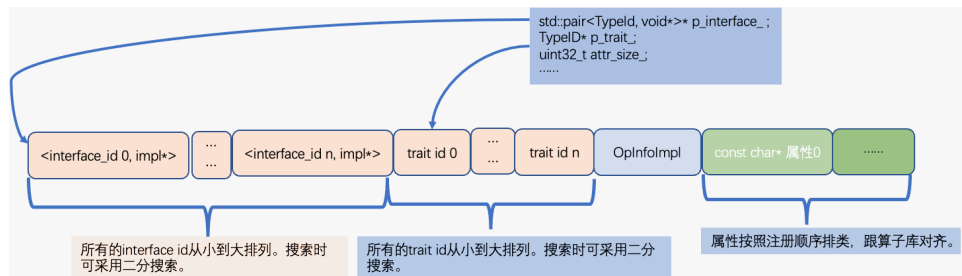
```

1 template <typename ConcreteOp>
2 static OpInfoImpl* create(); //根据具体的Op, 创建一个OpInfoImpl

```

Opinfo包含一个**OpInterfaceMap**和一个**OpTraitSet**, 记录该算子类型所支持的接口和特征, OpInfo提供接口, 用来判定相应的Operation是否支持某个接口和特征, 以及, 将该算子直接转换为相应的接口和特征进行使用。

Opinfo 内存排列



OP设计

算子定义的核心在于, 以该算子类型为模版参数, 能够配合OpInfoImpl::create函数构造出OpInfoImpl对象。

</>

C++ | 收起 ^

```

1 // Op最为所有算子的工具基类, 用来将算子和它的特征关联起来
2 // 第一个模版参数为ConcreteOpType, 表示具体的算子类型
3 // 后续的所有模版参数都是该算子所具有的特征和接口。
4 template <typename ConcreteOp, class... TraitOrInterface>
5 class Op : public OpBase {
6 public:

```

```

7 // 利用TraitOrInterface中是OpTraitBase还是OpTraitInterface的基类，分别拆出
  TraitList和InterfaceList.
8 using TraitList =std::tuple<...>
9 using InterfaceList =std::tuple<...>
10 }
11 template <typename ConcreteOp>
12 OpInfoImpl* OpInfoImpl::create(){
13     // 根据ConcreteOp中的TraitList和InterfaceList推断出interface列表以及trait
    列表
14     // 然后计算Trait的数量以及Interface的数量，进行内存开辟
15     // 然后逐个赋值
16 }
17 class ConvOp: Op<ConvOp, ReadOnlyTrait, InferShapeInterface> {
18     // ConvOp包含了InferShapeInterface，因此，必须定义inferShape成员函数.
19     // 在该算子注册的时候，会实例化InferShapeInterface::Strategy<ConvOp>，如果没有
    定义inferShape函数，会在编译时报错
20     const char* name() { return "conv_2d";}
21     static void inferShape();
22 }
23 class IrContext {
24 public:
25     template<typename ConcreteOp>
26     bool registerOp();
27 private:
28     std::unordered_map<TypeId, OpInfoImpl*> op_info_map_;
29 }
30
31 template<typename ConcreteOp>
32 bool IrContext::registerOp() {
33     TypeId type_id = TypeId::get(ConcreteOp);
34     assert(op_info_map_.find(type_id) == op_info_map_.end());
35     op_info_map_[TypeId::get(ConcreteOp)] =
        OpInfoImpl::create<ConcreteOp>();
36 }

```

program设计

</>

C++ | 收起 ^

```

1 class Program {
2     // 模型的参数
3     std::unordered_map<StrAttribtue, Variable*> prams;
4     // 模型的计算图
5     std::list<Operation*> ops;

```

```
6 }
```

Inplace 设计:

torch :

变量定义为 `vtensor`, `tensor` `vtensor` 有值语义, `tensor` 表示存在别名, 默认初始变量都是 `tensor`

pass 1: 通过 `pass` 函数尽可能转换为 `vtensor` 随后进行图变换 :

pass 2: 有条件的进行 `to_tensor` 和 `to_vtensor` 的消除

`inplaces` 算子通过算子属性标志, 在 `pass` 阶段转换为非 `inplace` 算子+ `overwrite` 算子。

paddle:

定义:

```
</> C++ | 收起 ^  
  
1 class ReluOp: Op<ReluOp, ReadOnlyTrait, HasValueSemanticsTrait, ...>  
  {...}  
2 class Relu_Op: Op<Relu_Op, InplaceTrait, ...> {...}  
3 class ReshapeOp: Op<ReshapeOp, ViewLikeTrait, ReadOnlyTrait, ...> {...}
```

`ReadOnlyTrait` 表示该算子不会修改它的操作数 (意味着不是 `inplace` 算子)。

`HasValueSemanticsTrait` 比 `ReadOnlyTrait` 更强一层, 表示该算子不仅不会修改它的操作数, 而且不会给操作数增加别名。

(不仅不是 `inplace` 算子, 而且不是 `view` 类算子)。

`InplaceTrait` 表示该算子是某个算子的 `inplace` 版本, 它的算子名一定以下划线结尾, 而且去掉下划线, 就是算子的非 `inplace` 版本。

`ViewLikeTrait` 表示该算子的输出是输入的别名。(目前暂定它的第一个输出是第一个输入的别名, 以后可以考虑将其升级为 `Interface`)

变换:

为了处理多个 `tensor` 对应同一块内存这种情况, 我们将张量操作数在 IR 上分为两种类型。一种是 `tensor`, 另一种是 `v_tensor` (value tensor)。

两种类型的实现完全一致, 可以互相构造 (即可以通过 `v_tensor` 为参数, 构造 `tensor`; 反之亦然)。

定义两个工具算子: `to_tensor`, `to_vtensor` 实现 `tensor` 和 `v_tensor` 的类型转换 (深拷贝)。

pass 1: 通过 `pass` 函数尽可能转换为 `vtensor` 随后进行图变换 :

pass 2: 有条件的进行 `to_tensor` 和 `to_vtensor` 的消除, 消除 `pass1` 中产生的不必要的深拷贝。eg `to_tensor` 的输出不会被修改或者别名, 则可用输入代替

